



AUDIT REPORT

June 2026

For



BELDEX

Table of Content

Executive Summary	03
Number of Security Issues per Severity	04
Summary of Issues	05
Checked Vulnerabilities	06
Techniques and Methods	08
Types of Severity	10
Types of Issues	11
Severity Matrix	12
 Low Severity Issues	13
1. Consider using Onwable2Step instead	13
 Informational Issues	14
2. Remove unnecessary comments before deployment	14
Functional Tests	15
Threat Model	16
Automated Tests	26
Closing Summary & Disclaimer	27



Executive Summary

Project Name	Beldex Token
Protocol Type	ERC20
Project URL	https://beldex.io/
Overview	BELDEX is a LayerZero OFT V2 omnichain ERC20 (9> decimals) that moves supply cross-chain by burning on the source and minting on the destination, with owner-gated mint, public self-burn.
Audit Scope	The scope of this Audit was to analyze the Beldex Token Smart Contracts for quality, security, and correctness.
Source Code link	https://bscscan.com/address/0xb5af6888af2415329bdb90924639b833d75ec927#code
Contracts in Scope	MyOFT.sol
Language	Solidity
Blockchain	BNB Smart Chain
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	19th June 2026

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	1 (50.0%)
Informational	1 (50.0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	1
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Consider using Onwable2Step instead	Low	Acknowledged
2	Remove unnecessary comments before deployment	Informational	Acknowledged



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Low Severity Issues

Consider using Ownable2Step instead

Acknowledged

Path

MyOFT.sol

Description

The project intends to use Ownable2Step semantics (safe 2-step ownership transfer), but currently inherits from Ownable, not Ownable2Step.

This means the owner can instantly transfer ownership in one transaction defeating the intended Two-Step security model documented in comments and project requirements.

If the owner key is compromised or misused, privileged control can be reassigned instantly without the beneficiary's acceptance.

Impact

Low

Likelihood

Low

Recommendation

Replace Ownable with Ownable2Step to enforce a safer two-step ownership transfer process



Informational Issues

Remove unnecessary comments before deployment

Acknowledged

Description

OFT contract contains comments that were probably used during testing phase. However it is necessary to remove them before deployment

```
1 // Uncomment the line below to mint test tokens on deployment (for testnet only) //@audit
2 // _mint(msg.sender, 100000 * (10 ** 18));
```

Recommendation

Consider removing the comments



Functional Tests

Some of the tests performed are mentioned below:

- ✓ decimals() returns 9
- ✓ name() and symbol() return the constructor-supplied values
- ✓ Owner succeeds and non-owner reverts
- ✓ Reverts on transferOwnership
- ✓ pause by owner sets paused() true and emits Paused
- ✓ unpause when not paused reverts (ExpectedPause)
- ✓ Burn succeeds again after unpause
- ✓ Deploy behavior when `_lzEndpoint == address(0)` (reverts in endpoint wiring)
- ✓ owner() returns the current owner



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Ownable (OpenZeppelin)	Library/ Base Contract	Single-owner access control for mint(), pause(), unpause(), and all OApp configuration setters	Correct ownership management; owner is trusted and competent	Single-step transfer can lead to permanent ownership loss; owner can mint unlimited supply
Pausable (OpenZeppelin)	Library/ Base Contract	Provides whenNotPaused gate on mint() and burn()	Mutex/state logic is correctly implemented	Paused state only blocks mint/burn, not send()/transfer()
ERC20 (OpenZeppelin, via OFT)	Library/ Base Contract	Implements token balances, transfer, approve, totalSupply	Correct ERC20 accounting	N/A
OFT / OFTCore (LayerZero)	Library/ Base Contract	Omnichain fungible token base; debit (burn) on send, credit (mint) on receive	Correct cross-chain accounting and decimal conversion	Debit/credit mismatch breaks supply conservation across chains



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OApp / OAppCore (LayerZero)	Library/ Base Contract	Manages peers, delegate, and endpoint messaging configuration	Peer and delegate are configured correctly	Misconfigured peer allows forged inbound mints
LayerZero Endpoint	External Contract	Sends and delivers cross-chain messages; calls lzReceive()	Endpoint is honest and delivers authentic messages only from configured peers	Compromised/ buggy endpoint can mint tokens via lzReceive
DVNs & Executor (LayerZero)	External Infrastructure	Verify and execute cross-chain message delivery	Configured DVNs are honest and non-colluding	Collusion/ compromise enables forged message delivery

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
MyOFT.sol	constructor()	What this function can do Initialize OFT name/symbol, wire the LayerZero endpoint, and set the delegate as owner



Contract Name	Function	Category
MyOFT.sol	decimals()	What this function cannot/should not do Mint an initial supply (commented out), assign ownership to an arbitrary unintended address Main invariant(s) Owner and delegate are set to _delegate at deployment Access level Implicit restriction (once at deployment) What this function can do Return the fixed local decimal precision of 9 What this function cannot/should not do Return a value below sharedDecimals (6), change at runtime Main invariant(s) decimals == 9, constant for the lifetime of the contract Access level Public, pure (anyone)
MyOFT.sol	renounceOwnership	What this function can do Always revert with DisabledRenounceOwnership What this function cannot/should not do Actually renounce ownership, leave the contract ownerless Main invariant(s) Ownership can never be renounced Access level Public, pure (anyone; always reverts)

Contract Name	Function	Category
MyOFT.sol	pause()	What this function can do Set the contract into paused state, blocking mint() and burn() What this function cannot/should not do Block send()/transfer()/lzReceive(), be called by a non-owner Main invariant(s) Only owner can pause; pause gates only mint/burn Access level onlyOwner
MyOFT.sol	unpause()	What this function can do Clear the paused state, re-enabling mint() and burn() What this function cannot/should not do Be called by a non-owner Main invariant(s) Only owner can unpause Access level onlyOwner
MyOFT.sol	mint(address _to, uint256 _value)	What this function can do Mint arbitrary amounts of tokens to any address and emit Mint What this function cannot/should not do Be called while paused, be called by a non-owner, mint to address(0) Main invariant(s) Only owner can increase supply; totalSupply increases by exactly _value



Contract Name	Function	Category
MyOFT.sol	burn(uint256 _value)	Access level onlyOwner, whenNotPaused What this function can do Burn the caller's own tokens and emit Burn What this function cannot/should not do Burn another account's tokens, be called while paused, burn more than the caller's balance Main invariant(s) Caller can only burn its own balance; totalSupply decreases by exactly _value
MyOFT.sol	send(SendParam, MessagingFee, address)	Access level Public, whenNotPaused (anyone, own balance only) What this function can do Debit (burn) the caller's tokens locally and dispatch a cross-chain message to credit them on the destination chain What this function cannot/should not do Send while the underlying ERC20 is restricted, debit more than allowed by minAmountLD/slippage, bypass dust removal Main invariant(s) amountSentLD on source equals amountReceivedLD on destination (1:1, minus dust); supply conserved across the mesh Access level Public, payable (anyone)



Contract Name	Function	Category
MyOFT.sol	lzReceive(Origin, bytes32, bytes, address, bytes)	What this function can do Receive a verified cross-chain message and credit (mint) tokens to the recipient What this function cannot/should not do Be called by anyone other than the LayerZero endpoint, accept messages from a non-configured peer, double-process a nonce Main invariant(s) Only the endpoint can invoke; messages must originate from the trusted peer for the source EID Access level External, restricted to LayerZero endpoint + peer check
MyOFT.sol	setPeer(uint32 _eid, bytes32 _peer)	What this function can do Configure the trusted remote OFT address for a given endpoint ID What this function cannot/should not do Be called by a non-owner, point to a malicious/incorrect peer Main invariant(s) Only owner sets peers; peers define the cross-chain trust set Access level onlyOwner



Contract Name	Function	Category
MyOFT.sol	setDelegate(address _delegate)	What this function can do Set the delegate authorized to configure the OApp inside the LayerZero endpoint What this function cannot/should not do Be called by a non-owner, grant endpoint configuration to an untrusted party Main invariant(s) Only owner sets the delegate Access level onlyOwner
MyOFT.sol	setMsgInspector(address _msgInspector)	What this function can do Set or disable the optional message/options inspector contract What this function cannot/should not do Be called by a non-owner, point to a faulty inspector that blocks valid sends Main invariant(s) Only owner sets the delegate Access level onlyOwner
MyOFT.sol	setEnforcedOptions(EnforcedOptionParam())	What this function can do Set enforced execution options per destination EID and message type What this function cannot/should not do Be called by a non-owner, set malformed options that brick sends Main invariant(s) Only owner configures enforced options



Contract Name	Function	Category
MyOFT.sol	transferOwnership(address newOwner)	Access level onlyOwner What this function can do Transfer contract ownership to a new address in a single step What this function cannot/should not do Transfer to address(0), be called by a non-owner Main invariant(s) Only current owner can transfer ownership
MyOFT.sol	transfer / transferFrom / approve	Access level onlyOwner What this function can do Standard ERC20 movement and allowance management of token balances What this function cannot/should not do Move more than balance/allowance, be gated by pause (these are NOT pausable) Main invariant(s) Sum of balances equals totalSupply; allowances respected
MyOFT.sol	quoteSend / quoteOFT	Access level Public (anyone, own balance/allowance) What this function can do Return the estimated LayerZero fee and the amount sent/received for a prospective send What this function cannot/should not do Mutate state, return values inconsistent with the actual send() path



Contract Name	Function	Category
		Main invariant(s) Quote matches the debit logic used in send() Access level External, view (anyone)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
MyOFT.sol	ERC20 (self)	The BELDEX token supply itself minted/burned by mint(), burn(), and cross-chain send()/lzReceive(); no third-party tokens are custodied



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
MyOFT.sol	mint	–	BELDEX (minted)	Owner	Unlimited supply inflation; no cap
MyOFT.sol	burn	BELDEX (burned)	–	Public	Caller-only burn; blocked while paused
MyOFT.sol	send	BELDEX (debited/ burned)	Cross-chain credit	Public	Decimal/dust conversion, slippage, supply conservation
MyOFT.sol	lzReceive	Cross-chain message	BELDEX (credited/ minted)	LayerZero Endpoint	Forged-peer mint, endpoint trust, replay
MyOFT.sol	transfer / transferFrom	BELDEX	BELDEX	Public	Not pausable moves even while paused



Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Beldex Token. We performed our audit according to the procedure described above.

No critical issues in Beldex Token, 1 Low and 1 Informational severity issues were found.

Beldex team Acknowledged both the issues

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**7+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1400+

Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



BELDEX



QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com